PEMROGRAMAN BERORIENTASI OBJEK 2026

Atribut dan *Method* Lebih Dalam

Data milik bersama, *method* tanpa objek, konstruktor luwes, dan apa yang sebenarnya disimpan variabel

PERTEMUAN

03

dari 16

Program Studi Informatika · Universitas Mulawarman
Jalur praktik: Python dan Java

Tujuan Pembelajaran

Setelah pertemuan ini, kamu mampu:

1. Membedakan **atribut objek** dan **atribut kelas**, lalu menentukan data mana yang cocok menjadi milik bersama.
2. Membedakan *method* objek dan *method* milik kelas (*static method*), termasuk kegunaan **konstruktor alternatif**.
3. Membuat konstruktor yang luwes dengan **nilai bawaan** (Python) atau ***overloading*** (Java).
4. Membuat **representasi teks** sebuah objek dan ***method boolean*** dengan nama yang mudah dibaca.
5. Menjelaskan bahwa variabel menyimpan **referensi**, lalu memprediksi akibat ***aliasing*** dan nilai kosong (`None`/`null`).
6. Memakai objek sebagai **parameter** dan sebagai **nilai kembalian** sebuah *method*.

Mengingat Kembali



Dari pertemuan sebelumnya

- **Kelas** adalah cetak biru, sedangkan **objek** adalah wujud nyatanya. Satu kelas bisa menghasilkan banyak objek.
- **Konstruktor** berjalan otomatis saat objek dibuat dan mengisi nilai awal atributnya.
- Setiap objek menyimpan atributnya sendiri. Menaikkan harga es teh tidak mengubah harga kopi.
- Variabel hanyalah “label” yang menunjuk ke objek. Waktu itu baru disinggung; hari ini kita bahas tuntas.

Agenda Hari Ini

1. Atribut objek dan atribut kelas
2. *Method* milik kelas dan konstruktor yang luwes
3. Objek yang bisa memperkenalkan diri
4. Variabel menyimpan referensi
5. Studi kasus KedaiKita

BAGIAN 1

1

Atribut Objek dan Atribut Kelas

Data milik setiap objek dan data milik bersama.

Buku Catatan dan Papan Pengumuman

Analogi

- Setiap mahasiswa membawa **buku catatan** sendiri. Ayu mencoret bukunya, buku Bima **tidak** ikut tercoret.
- **Papan pengumuman** hanya ada **satu** dan dibaca semua orang. Diganti sekali, semua langsung membaca info baru.
- "Jumlah peserta: 40 orang" tidak cocok ditulis di buku masing-masing karena menggambarkan **seluruh kelas**.

Di ruang kuliah	Di PBO
Buku catatan tiap mahasiswa	**Atribut objek**: `nim`, `nama`
Papan pengumuman di depan kelas	**Atribut kelas**: `kampus`, `jumlah`
Isi buku berbeda tiap orang	Satu nilai untuk setiap objek
Satu papan dibaca semua orang	Satu nilai dipakai bersama semua objek

Atribut Kelas di Python dan Java

Python menulis atribut kelas di badan kelas, di luar `__init__`. Java menandainya dengan kata kunci `static`.

Python python/contoh/contoh01_atribut_kelas.py

```python
class Mahasiswa:
    kampus = "Unmul"
    jumlah = 0

    def __init__(self, nim, nama):
        self.nim = nim
        self.nama = nama
        Mahasiswa.jumlah += 1

# ... di dalam main()
ayu = Mahasiswa("2409106001", "Ayu Lestari")
bima = Mahasiswa("2409106002", "Bima Saputra")
citra = Mahasiswa("2409106003", "Citra Dewi")
print(f"Jumlah mahasiswa: {Mahasiswa.jumlah}")
```

Java java/contoh/Contoh01AtributKelas.java

```java
class Mahasiswa {
    static String kampus = "Unmul";
    static int jumlah = 0;
    String nim;
    String nama;

    Mahasiswa(String nim, String nama) {
        this.nim = nim;
        this.nama = nama;
        Mahasiswa.jumlah += 1;
    }
}
// ... di dalam main
Mahasiswa ayu = new Mahasiswa("2409106001", "Ayu Lestari");
```

Jebakan: Mengubah Atribut Kelas lewat Objek

Setelah `ayu.masuk()` dan `bima.masuk()`	Python: `self.total += 1`	Java: `this.total += 1`
Total milik kelas	`0` (tidak berubah!)	`2`
`ayu.total`	`1` (atribut objek baru milik `ayu`)	`2` (nilai bersama)
Kejutannya	nilai bersama **tidak** bertambah	terlihat seperti milik objek, padahal milik bersama

Kebiasaan aman di kedua bahasa: **ubah atribut kelas selalu lewat nama kelasnya**, misalnya `Pengunjung.total += 1`.

BAGIAN 2

2

Method Milik Kelas dan Konstruktor yang Luwes

Method yang tidak butuh objek dan cara membuat objek dari data yang berbeda-beda.

Method Objek dan *Method* Milik Kelas

Pembeda	*Method* objek	*Method* milik kelas
Cara memanggil	`objek.method()`	`NamaKelas.method()`
Butuh objek?	ya	tidak
Punya `self`/`this`?	ya	tidak
Contoh	`kopi.hitung_total(2)`	`Tanggal.kabisat(2024)`
Cocok untuk	perilaku yang memakai data objek	fungsi bantu, konversi, pembuat objek

Python: `@staticmethod` atau `@classmethod`. Java: *method* `static`. Nama lainnya: *static method*.

Method Milik Kelas di Python dan Java

`kabisat()` cukup butuh angka tahun. `dari_teks()` adalah **konstruktor alternatif**: membuat objek `Tanggal` dari teks.

Python python/contoh/contoh03_static_dan_class_method.py

```python
class Tanggal:
    # ... (dilewati)
    @staticmethod
    def kabisat(tahun):
        # ... (dilewati)

    @classmethod
    def dari_teks(cls, teks):
        hari, bulan, tahun = teks.split("-")
        return cls(int(hari), int(bulan), int(tahun))

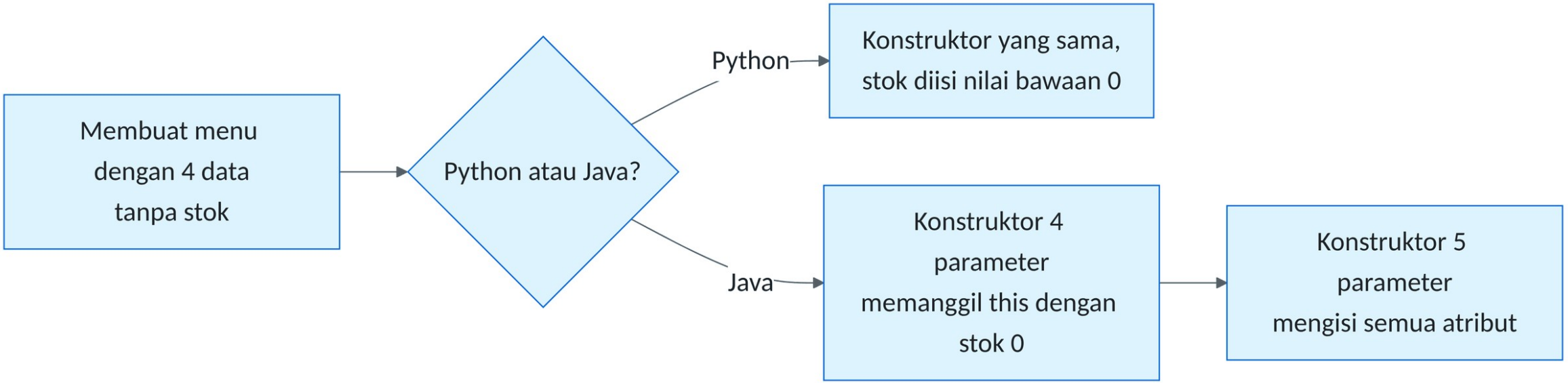
# ... di dalam main()
sumpah_pemuda = Tanggal.dari_teks("28-10-1928")
```

Java java/contoh/Contoh03MethodStatic.java

```java
class Tanggal {
    // ... (dilewati)
    static boolean isKabisat(int tahun) {
        // ... (dilewati)
    }

    static Tanggal dariTeks(String teks) {
        String[] bagian = teks.split("-");
        // ... (dilewati)
        return new Tanggal(hari, bulan, tahun);
    }
}
// ... di dalam main
Tanggal sumpahPemuda = Tanggal.dariTeks("28-10-1928");
```

Satu Konstruktor atau Beberapa?



Nilai bawaan (*default value*): nilai yang otomatis dipakai parameter jika argumennya tidak dikirim.

Nilai Bawaan dan *Overloading* Konstruktor

Python: satu konstruktor dengan nilai bawaan. Java: beberapa konstruktor yang meneruskan ke konstruktor lengkap.

Python python/contoh/contoh04_nilai_bawaan.py

```python
class Laptop:
    def __init__(self, merek, ram=8, penyimpanan=512):
        self.merek = merek
        self.ram = ram
        self.penyimpanan = penyimpanan

# ... di dalam main()
laptop_ayu = Laptop("Asus")
laptop_bima = Laptop("Lenovo", 16)
laptop_citra = Laptop("Acer", 16, 1024)
laptop_dimas = Laptop("HP", penyimpanan=256)
```

Java java/contoh/Contoh04OverloadingKonstruktor.java

```java
class Laptop {
    // ... (dilewati)
    Laptop(String merek, int ram, int penyimpanan) {
        // ... (dilewati)
    }
    Laptop(String merek, int ram) {
        this(merek, ram, 512);
    }
    Laptop(String merek) {
        this(merek, 8, 512);
    }
}
// ... di dalam main
Laptop laptopAyu = new Laptop("Asus");
Laptop laptopBima = new Laptop("Lenovo", 16);
```

BAGIAN 3

3

Objek yang Bisa Memperkenalkan Diri

Representasi teks dan *method* yang menjawab ya atau tidak.

Representasi Teks: `__str__` dan `toString()`

Dipanggil **otomatis** saat objek dicetak. Tugasnya **mengembalikan** teks, bukan mencetaknya.

Python python/contoh/contoh05_str.py

```python
class Motor:
    def __init__(self, plat, merek, warna):
        # ... (dilewati)

    def __str__(self):
        return f"{self.plat} - {self.merek} ({self.warna})"
# ... di dalam main()
motor_budi = Motor("KT 1234 AB", "Honda Beat", "merah")
print(motor_budi)
print(f"Motor Budi: {motor_budi}")
```

Java java/contoh/Contoh05ToString.java

```java
class Motor {
    // ... (dilewati)
    @Override
    public String toString() {
        return plat + " - " + merek + " (" + warna + ")";
    }
}
// ... di dalam main
System.out.println(motorBudi);
System.out.println("Motor Budi: " + motorBudi);
```

Output

```
KT 1234 AB - Honda Beat (merah)
Motor Budi: KT 1234 AB - Honda Beat (merah)
```

Method yang Menjawab Ya atau Tidak

Nama yang baik	Dibaca sebagai	Nama yang kurang baik
`stok_cukup(2)` / `stokCukup(2)`	"stoknya cukup untuk 2?"	`cek_stok(2)` (hasilnya apa?)
`kabisat(2024)` / `isKabisat(2024)`	"2024 kabisat?"	`tahun(2024)`
`saldo_cukup(50000)` / `saldoCukup(50000)`	"saldonya cukup?"	`proses_saldo(50000)`
`sudah_lunas()` / `sudahLunas()`	"sudah lunas?"	`status()`

Tidak perlu menulis `== True`: cukup `if menu.stok_cukup(2)` (Python) atau `if (menu.stokCukup(2))` (Java).

BAGIAN 4

4

Variabel Menyimpan Referensi

Janji dari Pertemuan 02: apa sebenarnya isi sebuah variabel?

Dua Nama, Satu yang Ditunjuk



Referensi adalah penunjuk ke objek. `ponsel_dipinjam = ponsel_budi` hanya menyalin penunjuknya; keduanya menunjuk objek yang sama (***aliasing***).

Aliasing dan Variabel tanpa Objek

Python memakai `is`, Java memakai `==` untuk bertanya: apakah kedua variabel menunjuk objek yang sama?

Python python/contoh/contoh06_referensi.py

```python
ponsel_budi = Ponsel("Budi", 80)
ponsel_dipinjam = ponsel_budi
ponsel_dipinjam.pakai(30)
# ... (dilewati)
if ponsel_budi is ponsel_dipinjam:
    print("Keduanya menunjuk objek yang SAMA")
# ... (dilewati)
ponsel_dimas = None
if ponsel_dimas is None:
    print("Dimas belum punya ponsel")
```

Java java/contoh/Contoh06Referensi.java

```java
Ponsel ponselBudi = new Ponsel("Budi", 80);
Ponsel ponselDipinjam = ponselBudi;
ponselDipinjam.pakai(30);
// ... (dilewati)
if (ponselBudi == ponselDipinjam) {
    System.out.println("Keduanya menunjuk objek yang SAMA");
}
// ... (dilewati)
Ponsel ponselDimas = null;
if (ponselDimas == null) {
    System.out.println("Dimas belum punya ponsel");
}
```

Mengirim Angka atau Mengirim Objek?

Mengirim angka

- *Method* menerima **salinan** nilainya.
- `tambah_bonus_angka(saldo_tunai)` menambah 5000 di dalam *method*.
- `saldo_tunai` milik pemanggil **tetap** 10000.

Mengirim objek

- *Method* menerima salinan **referensi**: objeknya sama.
- `tambah_bonus_dompet(dompet_siti)` menambah saldo objek itu.
- Saldo dompet Siti milik pemanggil **ikut** bertambah 5000.

Objek sebagai Nilai Kembalian

- Kalkulator menghitung `2 + 3` dan menghasilkan `5`. Angka `2` dan `3` tidak berubah; hasilnya angka **baru**.

- `total = kuliah.tambah(praktikum)` mengembalikan objek `Durasi` **baru**: 4 jam 10 menit.
- `kuliah` (2 jam 30 menit) dan `praktikum` (1 jam 40 menit) **tidak berubah**.
- Hasilnya bisa langsung dipakai lagi secara berantai: `kuliah.tambah(istirahat).tambah(praktikum)`.

Kiat

Dua gaya sama-sama sah: *method* yang **mengubah objeknya sendiri** atau yang **mengembalikan objek baru**. Pilih dengan sadar, lalu beri nama yang jujur.

STUDI KASUS

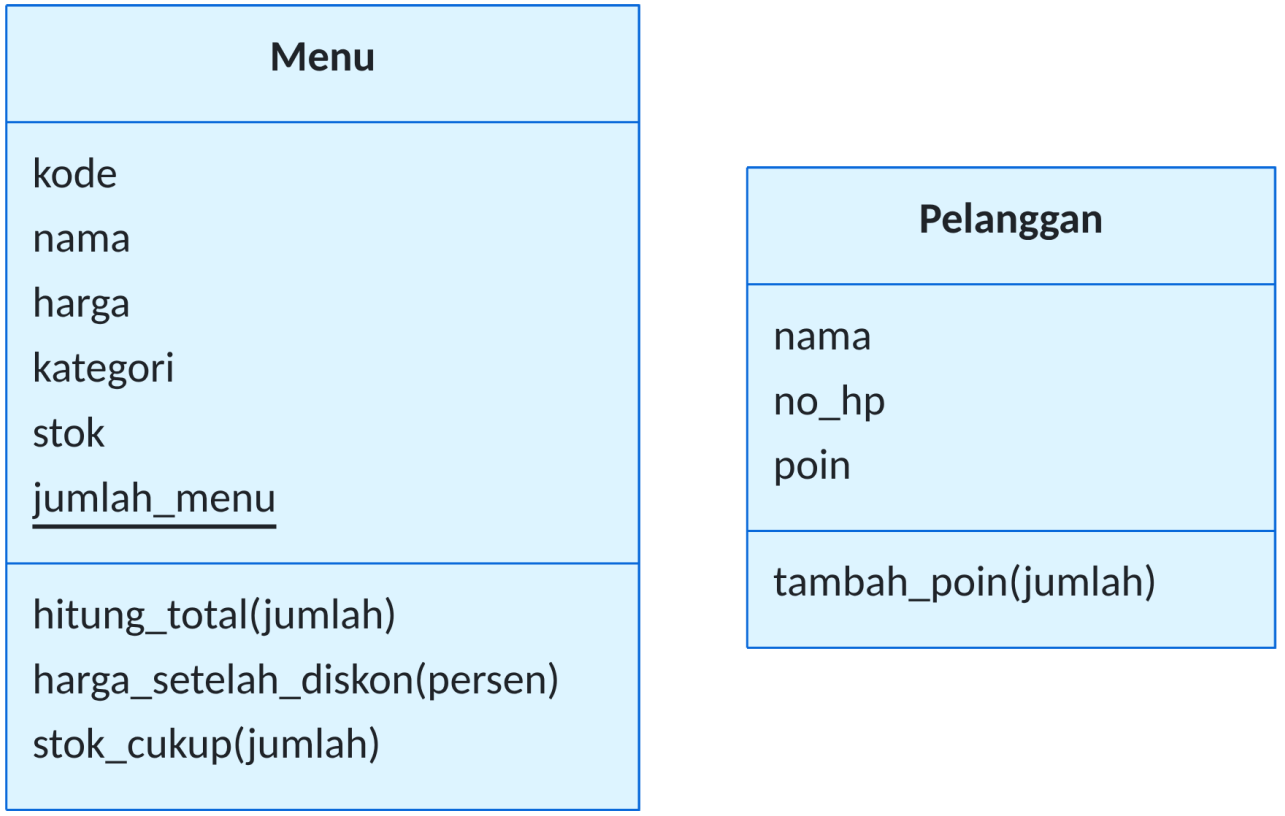


Studi Kasus KedaiKita

Menu yang lebih lengkap untuk kafe kecil di Jl. Pramuka, Samarinda.

Rancangan KedaiKita Pertemuan 03

Studi kasus KedaiKita



Atribut yang digarisbawahi (`jumlah_menu`) adalah atribut kelas. Di Java, nama memakai *camelCase*, misalnya `jumlahMenu`.

Apa yang Baru di KedaiKita?

Studi kasus KedaiKita

Perubahan	Konsep yang dipakai
Stok boleh tidak disebut saat membuat menu (otomatis 0)	nilai bawaan / *overloading* konstruktor
`jumlah_menu` mencatat banyaknya menu yang sudah dibuat	atribut kelas sebagai penghitung
Menu bisa langsung dicetak; `tampilkan_info()` tidak dipakai lagi	representasi teks
`harga_setelah_diskon(persen)` menghitung harga promo	*method* yang mengembalikan nilai
`stok_cukup(jumlah)` menjawab apakah stok mencukupi	*method boolean*
Uang ditampilkan dengan titik ribuan: `Rp18.000`	fungsi bantu / *method* milik kelas

Python memakai fungsi biasa `format_rupiah()`. Java tidak mengizinkan fungsi di luar kelas, jadi memakai kelas utilitas `Rupiah.format()`.

Kelas `Menu` Versi Pertemuan 03

Studi kasus KedaiKita

Potongan kelas `Menu`: atribut kelas sebagai penghitung, *method* yang mengembalikan nilai, dan *method boolean*.

Python python/studi-kasus/kedaikita.py

```python
class Menu:
    jumlah_menu = 0
    # ... (dilewati)
    def harga_setelah_diskon(self, persen):
        potongan = self.harga * persen // 100
        return self.harga - potongan

    def stok_cukup(self, jumlah):
        return self.stok >= jumlah
```

Java java/studi-kasus/KedaiKita.java

```java
class Menu {
    static int jumlahMenu = 0;
    // ... (dilewati)
    int hargaSetelahDiskon(int persen) {
        int potongan = harga * persen / 100;
        return harga - potongan;
    }

    boolean stokCukup(int jumlah) {
        return stok >= jumlah;
    }
}
```

Miskonsepsi yang Sering Terjadi

Anggapan keliru	Yang benar
"*Static method* bisa membaca atribut objek seperti `nama` atau `harga`."	Tidak bisa secara langsung karena tidak punya `self/this`. Jika butuh data objek, kirim objeknya sebagai parameter atau jadikan *method* objek biasa.
"`None/null` adalah objek kosong yang tetap bisa dipakai."	`None/null` berarti tidak ada objek. Memanggil *method* lewat variabel berisi `None/null` pasti menimbulkan *error*.
"*Method* tidak bisa mengubah objek yang dikirim kepadanya."	Bisa. *Method* menerima salinan referensi, jadi memegang objek yang sama dengan pemanggil. Yang tidak bisa: mengganti objek yang dipegang variabel pemanggil.

Kuis Cepat

Kelas `Mahasiswa` punya beberapa data. Manakah yang paling tepat dijadikan **atribut kelas** (milik bersama), bukan atribut objek?

A. NIM

B. Nama lengkap

C. Jumlah seluruh mahasiswa yang sudah terdaftar

D. IPK

Kuis Cepat: Jawaban

Kelas `Mahasiswa` punya beberapa data. Manakah yang paling tepat dijadikan **atribut kelas** (milik bersama), bukan atribut objek?

A. NIM

B. Nama lengkap

C. Jumlah seluruh mahasiswa yang sudah terdaftar

D. IPK

Jawaban: C

Jumlah seluruh mahasiswa menggambarkan **semua** objek sekaligus, bukan milik satu mahasiswa tertentu, jadi cocok menjadi atribut kelas (Java: `static`). NIM, nama, dan IPK berbeda untuk setiap mahasiswa sehingga tetap menjadi atribut objek. Jika NIM dijadikan atribut kelas, semua mahasiswa akan berbagi satu NIM yang sama.

Kuis Cepat

Kelas `Ponsel` punya atribut `baterai` dan *method* `pakai(persen)` yang mengurangi baterai sebanyak `persen`. Apa yang tercetak?

```
a = Ponsel("Budi", 80)
b = a
b.pakai(30)
print(a.baterai)
```

A `80`

B `50`

C `30`

D `110`

Kuis Cepat: Jawaban

Kelas `Ponsel` punya atribut `baterai` dan *method* `pakai(persen)` yang mengurangi baterai sebanyak `persen`. Apa yang tercetak?

A `80`

✓ `50`

C `30`

D `110`

Jawaban: B

`b = a` tidak membuat ponsel baru; `a` dan `b` menunjuk **objek yang sama** (*aliasing*). Memakai 30% lewat `b` mengurangi baterai objek itu, jadi dilihat lewat `a` pun hasilnya 50. Pilihan A adalah miskonsepsi "`b = a` membuat salinan", C adalah baterai yang dipakai, dan D muncul jika `pakai()` dikira menambah baterai.

Ringkasan

- **Atribut objek** milik setiap objek; **atribut kelas** (Java: `static`) dipakai bersama dan diubah lewat nama kelas.
- *Method* milik kelas dipanggil lewat nama kelas tanpa objek. **Konstruktor alternatif** membuat objek dari data lain.
- Konstruktor luwes: Python memakai **nilai bawaan**, Java memakai ***overloading*** dan `this(...)`.
- `__str__`/`toString()` **mengembalikan** teks yang mewakili objek dan dipanggil otomatis saat objek dicetak.
- ***Method boolean*** dinamai seperti pertanyaan ya/tidak, misalnya `stok_cukup()` atau `isKabisat()`.
- Variabel menyimpan **referensi**: `b = a` membuat ***aliasing***; `None`/`null` berarti tidak ada objek.
- Objek kiriman bisa diubah oleh *method*, sedangkan *method* bisa **mengembalikan objek baru** tanpa mengubah objek asal.

Langkah Berikutnya

- Jalankan contoh 01—08 dan studi kasus KedaiKita di jalur pilihanmu.
- Kerjakan latihan, lalu cek pemahamanmu dengan kuis 10 soal.

Buka bahan pertemuan ini di GitHub:

- Konsep
- Praktik Python
- Praktik Java
- Jembatan Python ↔ Java
- Latihan
- Kuis 10 soal

Pindai untuk membuka folder pertemuan ini di GitHub

github.com/antonprafanto/PBO2026/tree/master/Pertemuan-03-Atribut-dan-Method